

MÓDULO 1 · PIPELINE DE DATOS Y PREPARACIÓN PROFESIONAL

Ingeniería de Características

Material teórico — Feature Engineering, codificación y escalado

Objetivo del material

Comprender cómo transformar los datos originales en características útiles para un modelo de Machine Learning, aplicando técnicas de codificación, escalado, transformación y creación de nuevas variables.

1. ¿Qué es Feature Engineering?

La **Ingeniería de Características** o **Feature Engineering** es el proceso de transformar los datos disponibles para construir variables que permitan a un modelo de Machine Learning aprender de manera más efectiva.

Una característica o *feature* es una variable utilizada como entrada de un modelo.

Idea central

Los modelos no comprenden directamente el significado de conceptos como 'cliente Premium', 'lunes' o 'alto nivel de ingresos'. Debemos representar esa información de una forma que el algoritmo pueda procesar.

2. ¿Por qué es importante?

La calidad de las características puede tener un impacto significativo en el rendimiento de un modelo.

- Permite representar mejor el problema.
- Facilita que el modelo encuentre patrones.
- Puede mejorar el rendimiento predictivo.
- Puede reducir ruido o información irrelevante.
- Puede hacer más interpretables los resultados.
- Puede reducir la cantidad de variables necesarias.

3. Variables originales y variables derivadas

Una variable original es aquella que proviene directamente de la fuente de datos. Una variable derivada se construye a partir de una o más variables existentes.

Por ejemplo, si tenemos:

```
fecha_nacimiento
fecha_actual
```

podemos crear:

```
edad = fecha_actual - fecha_nacimiento
```

La nueva variable puede resultar más útil para el modelo que las fechas originales.

4. Tipos de variables

Tipo	Descripción	Ejemplo
Numérica	Representa cantidades o medidas.	Edad, ingresos, temperatura.
Catégorica	Representa categorías sin orden numérico.	Ciudad, color, tipo de cliente.
Ordinal	Categorías que poseen un orden.	Bajo, medio, alto.
Temporal	Representa fechas u horas.	Fecha de compra.
Texto	Contiene información textual.	Comentarios, reseñas.

5. Codificación de variables catégoricas

Muchos algoritmos de Machine Learning trabajan con valores numéricos. Por este motivo, las variables catégoricas suelen necesitar una transformación.

Método	Uso	Resultado
One-Hot Encoding	Variables catégoricas nominales.	Una columna por categoría.
Ordinal Encoding	Categorías con orden.	Cada categoría recibe un valor ordenado.
Label Encoding	Habitualmente utilizado para representar etiquetas.	Cada categoría recibe un número.

6. One-Hot Encoding

Supongamos una variable llamada **ciudad**:

```
Montevideo
Canelones
Maldonado
```

One-Hot Encoding puede generar una columna para cada categoría:

```
ciudad_Montevideo
ciudad_Canelones
ciudad_Maldonado
```

Una observación tendrá 1 en la categoría correspondiente y 0 en las demás.

En Scikit-learn:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown='ignore')
X_encoded = encoder.fit_transform(X)
```

7. Ordinal Encoding

Cuando las categorías tienen un orden natural, puede utilizarse una codificación ordinal.

```
Bajo → 1
Medio → 2
Alto → 3
```

Esta representación es adecuada cuando el orden tiene significado real.

No debe utilizarse indiscriminadamente para categorías nominales como colores o ciudades, porque podría introducir un orden que no existe.

8. Label Encoding

Label Encoding asigna un número a cada categoría. Es especialmente habitual para representar la variable objetivo cuando las clases son categóricas.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
```

Debe utilizarse con cuidado sobre variables de entrada, porque asignar números puede generar una relación de orden artificial entre categorías.

9. Escalado de variables

El escalado transforma las variables numéricas para que tengan magnitudes comparables.

Esto es especialmente importante para algoritmos que utilizan distancias o son sensibles a la magnitud de las variables.

Método	Característica	Cuándo utilizarlo
StandardScaler	Media 0 y desviación estándar 1.	Modelos sensibles a la escala.
MinMaxScaler	Transforma normalmente al rango 0–1.	Cuando interesa un rango definido.
RobustScaler	Utiliza estadísticas robustas.	Cuando existen outliers.

10. StandardScaler

StandardScaler estandariza las variables utilizando su media y desviación estándar.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Después de la transformación, las variables suelen quedar centradas alrededor de 0.

11. MinMaxScaler

MinMaxScaler transforma los valores para ubicarlos normalmente dentro de un intervalo entre 0 y 1.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

Es útil cuando interesa trabajar con un rango definido, aunque puede verse afectado por valores extremos.

12. RobustScaler

RobustScaler utiliza estadísticas robustas como la mediana y el rango intercuartílico.

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)
```

Puede ser una alternativa apropiada cuando existen valores atípicos importantes.

13. Creación de nuevas variables

Una de las tareas más importantes de Feature Engineering consiste en crear variables que representen mejor el problema.

Por ejemplo, a partir de:

```
ingresos
cantidad_personas_hogar
```

podemos crear:

```
ingreso_por_persona = ingresos / cantidad_personas_hogar
```

La nueva característica puede representar mejor la capacidad económica relativa del hogar.

14. Transformaciones matemáticas

Las transformaciones matemáticas pueden ayudar a representar mejor variables con distribuciones difíciles.

```
df['log_ingresos'] = np.log1p(df['ingresos'])
```

La transformación logarítmica puede reducir el efecto de valores extremadamente grandes y disminuir la asimetría de algunas distribuciones.

Debe aplicarse solamente cuando tenga sentido para la variable y teniendo en cuenta sus posibles valores.

15. Variables temporales

Una fecha completa puede contener mucha información que conviene separar en características independientes.

```
df['fecha'] = pd.to_datetime(df['fecha'])

df['anio'] = df['fecha'].dt.year
df['mes'] = df['fecha'].dt.month
df['dia'] = df['fecha'].dt.day
df['dia_semana'] = df['fecha'].dt.dayofweek
```

También pueden crearse variables como trimestre, fin de semana, hora del día o antigüedad desde una fecha determinada.

16. Binning o discretización

La discretización transforma una variable numérica continua en categorías o intervalos.

```
df['grupo_edad'] = pd.cut(
    df['edad'],
    bins=[0, 18, 30, 50, 100],
    labels=['Menor', 'Joven', 'Adulto', 'Mayor']
)
```

Esta técnica puede ser útil cuando el significado del problema se expresa mejor mediante rangos.

17. Interacciones entre variables

Una interacción representa una relación combinada entre dos o más variables.

Por ejemplo:

```
precio_por_m2 = precio / superficie
```

En otros problemas pueden crearse productos, razones, diferencias o combinaciones de variables.

18. Variables altamente sesgadas

Una variable sesgada presenta una distribución en la que los valores se concentran en una zona y existe una cola hacia valores más altos o más bajos.

Una transformación logarítmica puede ser útil en algunas variables positivas con fuerte asimetría.

```
np.log1p(df['ingresos'])
```

La decisión debe comprobarse mediante análisis exploratorio y conocimiento del dominio.

19. Feature Engineering y Data Leakage

Las nuevas características también pueden generar **data leakage** si utilizan información que no estaría disponible en el momento real de realizar una predicción.

Ejemplo

Si queremos predecir si un cliente abandonará el servicio el próximo mes, no podemos utilizar una variable que registre una acción realizada después de ese abandono.

20. Feature Engineering dentro de un Pipeline

Las transformaciones deben integrarse al flujo de Machine Learning para evitar diferencias entre entrenamiento y predicción.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

En proyectos con variables numéricas y categóricas diferentes puede utilizarse **ColumnTransformer** para aplicar transformaciones específicas a cada grupo.

21. ColumnTransformer

ColumnTransformer permite definir diferentes transformaciones para diferentes columnas.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), columnas_numericas),
    ('cat', OneHotEncoder(handle_unknown='ignore'), columnas_categoricas)
])
```

Esta estructura resulta especialmente útil en datasets reales donde conviven diferentes tipos de variables.

22. Errores frecuentes

- Codificar todas las variables categóricas de la misma manera.

- Aplicar Label Encoding a categorías sin orden.
- Escalar antes de separar entrenamiento y prueba.
- Crear variables utilizando información futura.
- Generar demasiadas características sin evaluar su utilidad.
- No considerar el significado de las variables.
- Crear variables que duplican información existente.
- Ignorar valores atípicos y distribuciones.
- No reproducir las transformaciones en producción.

23. Buenas prácticas profesionales

- Comprender el problema de negocio antes de crear variables.
- Analizar las variables mediante EDA.
- Documentar cada transformación.
- Separar entrenamiento y prueba antes de aprender parámetros.
- Utilizar Pipelines cuando sea posible.
- Evitar transformaciones arbitrarias.
- Comparar el rendimiento antes y después.
- Mantener solamente características justificadas.
- Verificar que las variables estén disponibles en producción.

24. Caso práctico: predicción de abandono

Supongamos que queremos predecir si un cliente abandonará un servicio.

Variables originales:

- edad
- ingresos
- fecha_alta
- cantidad_llamadas_soporte
- tipo_plan
- cantidad_meses

Podemos crear:

- antigüedad_meses
- llamadas_por_mes
- ingreso_por_mes
- es_plan_premium
- cliente_mayor_50

Estas nuevas variables pueden representar de forma más directa algunos factores relacionados con el abandono.

25. Flujo profesional de Feature Engineering

- 1. Comprender el problema.
- 2. Explorar los datos.
- 3. Identificar tipos de variables.
- 4. Detectar problemas de calidad.
- 5. Seleccionar transformaciones.
- 6. Crear características relevantes.
- 7. Codificar variables categóricas.
- 8. Escalar cuando corresponda.
- 9. Evitar data leakage.
- 10. Evaluar el impacto de las características.
- 11. Documentar el proceso.

26. Idea central

Feature Engineering convierte datos en información útil para un modelo.

No consiste simplemente en crear muchas variables. Consiste en representar el problema de una forma que permita al algoritmo aprender patrones relevantes, evitando ruido, sesgos y fuga de información.

27. Glosario

Concepto	Definición
Feature	Característica o variable utilizada como entrada de un modelo.
Feature Engineering	Proceso de crear o transformar características para mejorar la representación del problema.
One-Hot Encoding	Transformación que representa categorías mediante columnas binarias.
Label Encoding	Asignación de números a categorías.
Ordinal Encoding	Codificación que conserva un orden significativo entre categorías.
StandardScaler	Transformación que estandariza las variables alrededor de una media 0.
MinMaxScaler	Transformación que lleva los valores a un rango definido.
RobustScaler	Escalador menos sensible a valores extremos.
Data Leakage	Uso de información que no debería estar disponible durante el entrenamiento.
Pipeline	Secuencia reproducible de transformaciones y modelado.
ColumnTransformer	Herramienta para aplicar transformaciones diferentes a grupos de columnas.